

对状态的不完全观测截图详解

为什么单看当前画面不够，以及为什么要用 RNN 记忆过去

2026 年 4 月 9 日

目录

1 真实问题：这两张图到底在解决什么	2
2 出场对象：先把符号翻译成成人话	2
3 第一张图：为什么“不完全观测”会让决策变难	3
3.1 图 11.1 到底在画什么	3
3.2 三种观测层次的直观对比	3
3.3 最小贯穿例子：走迷宫时到底差在哪	3
3.4 第一张图真正想说明什么	4
4 为什么不能只看当前观测	4
5 第二张图：为什么要把观测历史作为输入	5
5.1 这一页到底在往前推进什么	5
5.2 这到底比原来多了什么	5
5.3 为什么这会立刻带来一个新困难	6
6 为什么 RNN 正好适合这里	7
6.1 补充解释：课本图 11.2 到底在画什么	8
6.2 用一个评论分类例子读懂图 11.2	8
6.3 补充解释：课本图 11.3 和图 11.4 到底在算什么	9
6.4 继续用同一个例子读懂图 11.3	10
7 对比：这两张图没有在讲什么	10
8 一句话总结	11

1 真实问题：这两张图到底在解决什么

这两张图真正想解决的，不是“怎么再引入一个新网络”，而是一个非常现实的问题：

如果智能体当前只能看到环境的一部分，它该怎么做决策？

在很多强化学习教材里，我们一开始都默认“状态 s_t 是完全可见的”。也就是说，智能体只要看一眼当前状态，就已经知道做决策所需的全部信息。

但真实任务常常不是这样。比如：

- 在迷宫里，你可能只看得到自己附近一小块区域；
- 在电子游戏里，屏幕上的画面只是完整游戏状态的一部分；
- 在扑克、麻将这类问题里，你从来都看不到全部信息。

因此，这两张图的核心问题可以压成一句话：

既然当前观测不完整，那么决策时除了看现在，还必须想办法利用过去看到过的东西。

2 出场对象：先把符号翻译成人话

在这两张图里，最重要的符号其实只有四个：

- s_t ：时刻 t 的真实状态，也就是环境“实际上”是什么样；
- o_t ：时刻 t 的当前观测，也就是智能体这时真正看见的东西；
- a_t ：时刻 t 要采取的动作；
- $o_{1:t}$ ：从开始到时刻 t 为止看过的全部观测历史。

所以，这里最关键的区分是：

- s_t 是环境的真实全貌；
- o_t 只是智能体此刻眼前能看到的一小部分。

而整章后面的想法，就是要回答：

既然 o_t 不够，那能不能把 $o_1, o_2, \dots, o_t$ 一起拿来，弥补当前观测的不完整？

3 第一张图：为什么“不完全观测”会让决策变难

3.1 图 11.1 到底在画什么

第一张图用迷宫举了三个层次的例子：

- 图 11.1(a)：完全观测。智能体能看到整个迷宫；
- 图 11.1(b)：不完全观测。智能体只能看到自己附近一小块区域；
- 图 11.1(c)：记忆过去观测。智能体把过去看过的局部区域也记下来，于是掌握的信息越来越多。

这三张图不是在画三种不同迷宫，而是在画：

同一个任务里，智能体掌握信息的多少不同，决策难度会有多大差别。

3.2 三种观测层次的直观对比

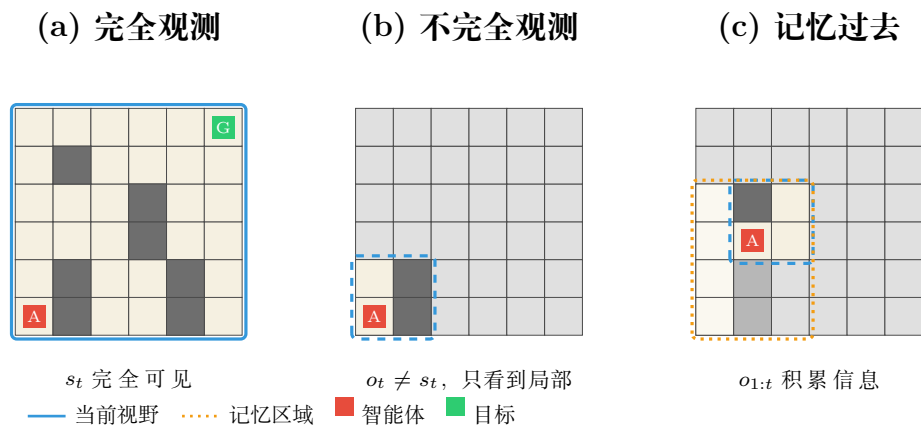


图 1: 迷宫中三种观测层次的对比：(a) 完全观测下 s_t 直接可见；(b) 不完全观测下只能看到局部 o_t ；(c) 将历史观测 $o_{1:t}$ 记住后，信息逐渐逼近 s_t 。

3.3 最小贯穿例子：走迷宫时到底差在哪

想象你在走迷宫，目标是找到出口。

情况 1：你能看到整个迷宫。 这就是图 11.1(a)。此时你一眼就能知道：

- 自己在哪里；
- 哪些路是死路；
- 哪条路线更接近出口。

这时做决策相对容易，因为当前看到的东西，已经几乎等于问题本身。

情况 2：你只能看到自己脚边一小块。 这就是图 11.1(b)。此时你虽然看到了局部墙壁，但你并不知道：

- 自己在整个迷宫的哪个位置；
- 前面这条走廊是不是你刚刚走过的；
- 左转和右转到底哪边更接近出口。

所以，仅靠当前观测 o_t 做决策时，智能体会非常“盲”。

情况 3：你把过去看过的区域也记下来。 这就是图 11.1(c)。虽然你现在依然只看得到局部，但你不再只有这一小块信息，因为你还记得：

- 你刚才从哪里来；
- 哪些地方已经走过；
- 哪些方向曾经碰到过死路。

因此，哪怕当前观测依旧不完整，只要你把历史整合起来，你手里的信息就会更接近真实状态。

3.4 第一张图真正想说明什么

第一张图的重点不是“迷宫”本身，而是下面这条逻辑：

1. 在完全观测问题里，当前状态 s_t 直接可见；
2. 在不完全观测问题里，智能体只能看到 o_t ，而 $o_t \neq s_t$ ；
3. 如果只把 o_t 喂给策略网络，决策常常不够好；
4. 因此，需要把过去观测也利用起来。

这就是为什么图中会先写出“直接用 $\pi(a_t | o_t; \theta)$ 做决策效果不好”，然后再把注意力转向“记住过去”。

4 为什么不能只看当前观测

这一步特别容易被忽略，所以单独说清楚。

如果当前观测已经足够，那当然没必要记忆过去。问题正是：

同一个局部画面 o_t ，可能对应多个不同的真实状态 s_t 。

还是拿迷宫举例。你现在眼前看到的，可能只是：

- 前面有墙；
- 左边有路；
- 右边也有路。

但这幅局部画面，完全可能在迷宫里的很多位置都出现过。也就是说：

只看 o_t ，你无法判断“我现在到底处在哪个全局位置”。

所以问题不是“当前观测没信息”，而是“当前观测的信息不够唯一”。

5 第二张图：为什么要把观测历史作为输入

5.1 这一页到底在往前推进什么

第二张图是在回答一个更具体的问题：

既然只看 o_t 不够，那么策略网络到底该吃什么输入？

作者给出的直接想法是：

$$o_{1:t} = [o_1, o_2, \dots, o_t].$$

这条式子不要急着当公式看，它首先是在说一句非常直白的话：

不要只把“现在看到的”交给策略网络，而要把“从开头到现在看过的全部内容”都交给它。

于是策略就不再写成

$$\pi(a_t \mid o_t; \theta),$$

而改成

$$\pi(a_t \mid o_{1:t}; \theta).$$

5.2 这到底比原来多了什么

原来的策略只看当前一帧：

$$\pi(a_t \mid o_t; \theta).$$

新的策略看的是完整观测历史：

$$\pi(a_t \mid o_{1:t}; \theta).$$

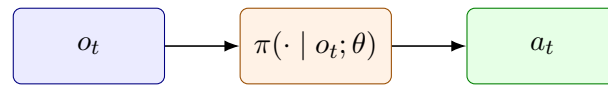
两者的区别，不只是输入变长了，而是决策依据变了：

- 前者问的是：“仅根据我现在眼前这一小块，我该做什么？”
- 后者问的是：“根据我一路走来看到的全部痕迹，我现在该做什么？”

因此，第二张图真正推进的是：

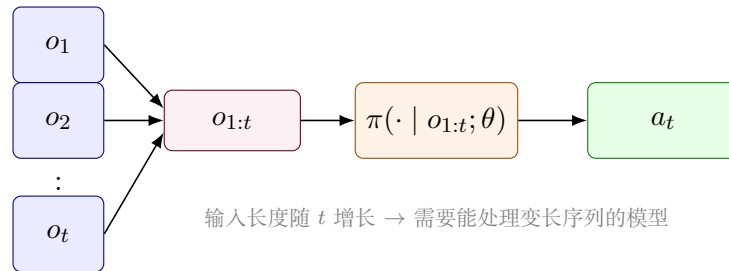
从“基于当前观测做决策”，转向“基于观测历史做决策”。

方案 A：仅看当前



信息可能不足以区分不同 s_t

方案 B：利用完整历史



输入长度随 t 增长 → 需要能处理变长序列的模型

图 2: 方案 A 仅将当前观测 o_t 送入策略网络；方案 B 将完整观测历史 $o_{1:t}$ 作为输入，信息更丰富但输入长度可变。

5.3 为什么这会立刻带来一个新困难

这正是第二张图后半部分想解释的地方。

如果 $o_1, \dots, o_t$ 都是同样维度的向量，那么

$$o_{1:t}$$

的长度会随着 t 不断增长。

也就是说：

- 在第 1 步，输入很短；
- 在第 10 步，输入更长；
- 在第 100 步，输入又更长。

于是立刻出现一个工程问题：

普通全连接层和普通卷积层通常要求输入长度固定，但这里的观测历史长度是不断变化的。

所以，这页真正想说的是：

1. 为了解决不完全观测，最自然的做法是记住历史；
2. 但一旦记历史，输入就变成了可变长度；
3. 因此，普通固定输入长度的网络不再合适；
4. 这就为 RNN 出场创造了理由。

6 为什么 RNN 正好适合这里

到这里，RNN 的角色就不再神秘了。

RNN 之所以适合这个问题，不是因为“它很高级”，而是因为它天然擅长处理：

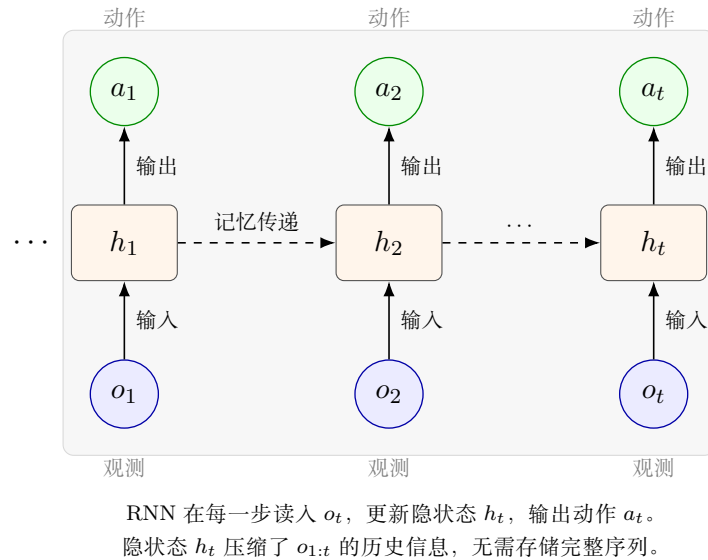


图 3: RNN 处理观测序列的展开示意：每一时刻读入观测 o_t ，通过隐状态 h_t 累积历史信息，再输出动作 a_t 。

- 一个一个到来的输入；
- 长度不固定的历史；
- 需要把过去信息压缩进一个内部状态里的任务。

你可以把它理解成一种“在线记笔记”的机制：

- 新看到一个观测 o_t ，就更新一次内部记忆；
- 不需要把全部历史原样摊开存储；
- 只需要把“对决策最重要的那部分过去信息”压进当前状态。

所以，这两张图连起来的完整逻辑是：

1. 当前观测 o_t 往往不足以代表真实状态 s_t ；
2. 因此必须利用过去的观测历史；
3. 但完整观测历史 $o_{1:t}$ 的长度在变；
4. 因而需要能处理变长序列、并能积累历史信息模型；
5. RNN 正是为这类场景准备的。

6.1 补充解释：课本图 11.2 到底在画什么

如果把课本里的图 11.2 当成一个黑盒，它其实只在表达三件事：

1. 输入不是一次性塞进去，而是按顺序一个个进入：先读 x_1 ，再读 x_2 ，一直读到 x_n ；
2. 每读到一个新输入，RNN 都会更新一次自己的内部记忆，于是得到新的 h_t ；
3. 这个 h_t 不是“只描述当前 x_t ”，而是“概括从 x_1 到 x_t 这一整段历史”。

所以，图里画出 $h_1, h_2, \dots, h_n$ ，不是为了说“网络在重复做同一件无聊的小事”，而是为了强调：

记忆是在一步一步积累的。越往后的 h_t ，通常掌握的信息越完整。

这也是为什么很多分类任务只取最后一个 h_n 去做预测。因为当序列已经全部读完时， h_n 才最像对整段输入的总摘要。

6.2 用一个评论分类例子读懂图 11.2

沿用课本里的场景。假设一条商品评论只有三个词：

“电池 / 不 / 耐用”

如果把 RNN 想成一个一路做笔记的人，那么它的内部变化可以直观地理解为：

- 读到“电池”时， h_1 只知道话题和电池有关，还不知道评价是好是坏；
- 读到“不”时， h_2 开始意识到语气可能要转向负面，但信息仍不完整；
- 读到“耐用”时， h_3 才能把前后的关系合起来，理解成“电池不耐用”，于是形成比较明确的负面判断。

这时，把最后的 h_3 送进分类器，输出的正面概率 $\hat{p}$ 就应该比较低。

这个例子想强调的不是具体数字，而是一个顺序关系：

同样是三个词，只有按顺序读完，最后的状态才知道整句话真正想表达什么。

这也顺便解释了图 11.2 的优缺点：

- 好处是它能把变长序列压缩成固定大小的状态，方便后续分类或决策；
- 局限是这个状态容量有限，序列很长时，前面较早的信息可能被后面的内容冲淡。

6.3 补充解释：课本图 11.3 和图 11.4 到底在算什么

如果说图 11.2 解释的是“RNN 在做什么”，那么图 11.3 解释的就是：

RNN 到底怎样把“旧记忆”和“新输入”合成为“新记忆”。

课本里的公式

$$h_t = \tanh(W[h_{t-1}; x_t] + b)$$

可以按下面三步来理解。

第一步：把旧信息和新信息放到一起。 $[h_{t-1}; x_t]$ 的意思不是做什么神秘操作，而只是把两部分信息并排放好：

- h_{t-1} 回答“我之前已经记住了什么”；
- x_t 回答“我这一刻又新看到了什么”。

第二步：按照规则重新混合。 矩阵 W 可以把它看成一张“混合规则表”：

- 每一列对应旧记忆或当前输入里的某一个分量；
- 每一行对应新记忆 h_t 里的某一个分量；
- 正权重表示把该信息往正方向推动；
- 负权重表示把该信息往反方向推动；
- 绝对值越大，说明影响越强。

而偏置 b 可以理解成默认基线或门槛：即使没有很强的新证据，系统也会先有一个默认倾向。

第三步：把结果压到合理范围。 $\tanh$ 对应图 11.4 里的 S 形曲线。它的作用不是增加新信息，而是把过大的值压回到 -1 到 1 之间。

直观上，它表达的是：

- 输入很大且为正时，输出接近 1 ，表示“强烈正向”；
- 输入很大且为负时，输出接近 -1 ，表示“强烈负向”；
- 输入接近 0 时，输出也接近 0 ，表示“态度还不明显”。

6.4 继续用同一个例子读懂图 11.3

为了把图 11.3 读得更直观，我们暂时把隐藏状态想成一个单独的“情绪分数”：

- 正数表示更偏正面；
- 负数表示更偏负面；
- 数值绝对值越大，表示判断越强。

注意，这只是为了帮助理解。真实的 h_t 往往是一个向量，不只记录情绪，还会记录主题、语气、否定关系等多种信息。

现在继续看评论“电池 / 不 / 耐用”：

- 读完“电池”后，旧记忆可能还是接近 0，因为只知道主题，还不知道倾向；
- 读到“不”时，矩阵 W 会把这个词当成强烈的反向信号，线性组合的结果可能变成 -1.2 ；
- 再经过 $\tanh$ 后，输出大约变成 -0.83 ，表示新的记忆已经明显偏负面；
- 接着读到“耐用”时，网络不会把它当成孤立的正面词，而会和前面的“不”一起理解，于是最终状态仍保持负面。

这组数字本身并不重要，重要的是图 11.3 想传达的因果链：

先把过去记忆和当前输入合在一起，再按参数设定的重要性重新加权，最后用 $\tanh$ 把结果压成稳定的新记忆。

所以，图 11.2 和图 11.3 其实是一前一后的关系：

- 图 11.2 在讲宏观流程：序列怎样一步步变成最后的摘要；
- 图 11.3 在讲微观机制：每一步的摘要究竟是怎样更新出来的。

7 对比：这两张图没有在讲什么

为了防止误读，再明确两点。

- 这两张图不是在说“只要把历史都记住，就一定解决问题”。它们只是说明：在不完全观测场景里，不记历史几乎一定不行。
- 这两张图也不是在证明“RNN 永远最好”。它们只是从问题结构出发，说明“为什么像 RNN 这样的记忆模型会自然出现”。

8 一句话总结

如果把这两张图压缩成一句最短的话，那就是：

在不完全观测问题里，当前看到的 o_t 不等于真实状态 s_t ，所以决策必须结合过去观测；而一旦输入变成可变长度的观测历史 $o_{1:t}$ ，RNN 这类能累积记忆的模型就变得自然且必要。